Reviewer Pack — Minimal Rank of Affine Root Systems in Representation Theory…

Entry 5c9fa7d81763 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 07:42:16 UTC

Minimal Rank of Affine Root Systems in Representation Theory vs Permutation Circuit Size

Entry ID: `5c9fa7d81763` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 07:42:16 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory **Field B** (complexity object): Permutation Circuit Complexity

Statement:

> ['For a given permutation, its corresponding affine root system has a minimal rank that is polynomial in the size of the permutation.', 'For all permutations π with length n, there exists an integer r such that the minimal rank of the affine root system associated with π is at most O(n^r).', 'The size of the smallest permutation circuit computing π is upper bounded by the minimal rank of its associated affine root system.']

Rationale (proposer's reasoning):

> ['Affine root systems provide a natural algebraic structure for representing permutations, as they capture the symmetries and invariants of the permutation group.', 'Previous work has shown that representation theory can be used to analyze the complexity of certain combinatorial problems.', 'This conjecture suggests that affine root systems could provide insights into the complexity of permutation circuits, potentially leading to new lower bounds.']

Taxonomy category: `Representation Theory × Permutation Circuit Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5e45d91915f7ab6a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the ratio between the minimal rank of an affine root system and the length of its associated permutation is less than or equal to a polynomial threshold, specifically O(n^r) where r ≤ 2, for at least 80% of the permutations tested. The conjecture is falsified if this ratio exceeds O(n^r) for any permutation.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - minimal rank affine root systems representation theory AND permutation circuit complexity - affine root system minimal rank polynomial size permutation representation theory OR permutation circuit complexity - permutation circuit size upper bounded minimal rank affine root system representation theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def factorial(n):
        if n == 0 or n == 1:
            return 1
        result = 1
        for i in range(2, n + 1):
            result *= i
        return result

    def gcd(a, b):
        while b != 0:
            a, b = b, a % b
        return a

    def lcm(a, b):
        return abs(a * b) // gcd(a, b)

    def matrix_multiply(A, B):
        m = len(A)
        n = len(B[0])
        p = len(B)
        result = [[0 for _ in range(n)] for _ in range(m)]
        for i in range(m):
            for j in range(n):
```

```
            for k in range(p):
                result[i][j] += A[i][k] * B[k][j]
    return result

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i + 1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, n):
            factor = A[j][i] / A[i][i]
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0 for _ in range(n)]
    for i in range(n - 1, -1, -1):
        x[i] = b[i]
        for j in range(i + 1, n):
            x[i] -= A[i][j] * x[j]
        x[i] /= A[i][i]
    return x

def permutation_circuit_size(perm):
    n = len(perm)
    circuit = [0] * n
    for i in range(n):
        if perm[i] != i:
            j = i
            while perm[j] != j:
                circuit[perm[j]] += 1
                j = perm[j]
            circuit[perm[j]] += 1
    return max(circuit)

def affine_root_system_rank(perm):
    n = len(perm)
    A = [[0 for _ in range(n)] for _ in range(n)]
    b = [0 for _ in range(n)]
    for i in range(n):
        A[i][i] = 1
        b[i] = perm[i]
    x = gaussian_elimination(A, b)
    rank = sum(1 for val in x if abs(val) > 1e-9)
    return rank

def generate_random_permutation(n):
    elements = list(range(n))
    random.shuffle(elements)
    return elements

n_min = 5
n_max = 40
```

```
    instances_per_seed = 30
    total_instances = (n_max - n_min + 1) * instances_per_seed

    metric_values = []
    conjecture_holds_count = 0

    for n in range(n_min, n_max + 1):
        for _ in range(instances_per_seed):
            perm = generate_random_permutation(n)
            rank = affine_root_system_rank(perm)
            circuit_size = permutation_circuit_size(perm)
            metric_values.append(rank / n)

    mean_metric_value = sum(metric_values) / total_instances
    std_metric_value = math.sqrt(sum((x - mean_metric_value) ** 2 for x in metric_values) / total_
    support_fraction = sum(1 for val in metric_values if val <= 2 * n_min) / len(metric_values)

    conjecture_holds = support_fraction >= 0.8

    return {
        "metric_name": "Ratio of Minimal Rank to Permutation Length",
        "metric_value": mean_metric_value,
        "instances_tested": total_instances,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"Ratio exceeded 2 * n_min for some permutat
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30))

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractior
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample='Ratio exceeded 2 * n_min' first_failing_seed={fi
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test timed out before producing data, which means we cannot verify the conjecture's support or falsification based on the pre-registered criteria. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14079
2	preregistration	ollama_remote	glm4:latest	0	0	10344
3	novelty	ollama_remote	glm4:latest	0	0	8465
4	novelty	ollama_remote	glm4:latest	0	0	9028
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11522
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6404
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9115
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14172
9	judge	ollama_remote	glm4:latest	0	0	12712

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 95842 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5c9fa7d81763.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5c9fa7d81763.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5c9fa7d81763.tar.gz` (if generated)